TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A Major Project Proposal

On

REAL-TIME DETECTION OF SYNTHETIC SPEECH IN VoIP COMMUNICATION SYSTEMS

Submitted By:

Bibek Joshi	(THA079BEI007)
Dharmananda Joshi	(THA079BEI009)
Neev Badu	(THA079BEI015)
Ujjwal Dahal	(THA079BEI046)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

June 2026

ACKNOWLEDGEMENT

We would like to thank the Institute of Engineering, Tribhuvan University, for incorporating our major project within the curriculum of Bachelor's in Electronics, Communication, and Information Engineering.

We would like to express our sincere gratitude to the **Department of Electronics and Computer Engineering, Thapathali Campus,** for providing us with the opportunity to carry out this project. We are especially thankful to **Er. Saroj Shakya** for his continuous guidance, constructive suggestions, and encouragement during the initiation of this project. His insights have been invaluable in refining the problem statement, methodology, and overall direction of the work.

We also acknowledge the support of the friends, faculty members of the department for their helpful discussions and academic guidance during the project initiation phase.

Bibek Joshi (THA079BEI007)
Dharmananda Joshi (THA079BEI009)
Neev Badu (THA079BEI015)
Ujjwal Dahal (THA079BEI046)

ABSTRACT

The rapid advancement of artificial intelligence based text-to-speech, voice conversion, and voice cloning technologies has made synthetic speech increasingly realistic and difficult to distinguish from genuine human speech, creating serious risks for VoIP communication, voice authentication, online meetings, call center security, fraud prevention, and digital trust. This proposal presents the design and evaluation of a real-time Audio Deepfake Detection system for identifying AI-generated speech in degraded Voice over Internet Protocol and real-time communication environments. Unlike conventional approaches that mainly evaluate detection models on clean offline recordings, the proposed project focuses on receiver-side reconstructed audio streams affected by practical communication processes such as codec compression, packetization, network delay, jitter, packet loss, packet loss concealment, decoding, buffering, and audio enhancement. The system will process short streaming audio chunks, apply preprocessing and feature extraction using speech representations such as MFCC, LFCC, CQCC and self-supervised speech embeddings, and classify each segment as real or synthetic using a machine learning model like AASIST and XGBoost. Temporal aggregation will be used to stabilize chunk-level predictions and generate confidence scores suitable for real-time alerts, monitoring, and security enforcement. The detector is proposed as an external API-based inference service that can be integrated with RTC or VoIP platforms through client-side or server-side deployment without modifying the core communication stack. The project will also implement realistic RTC system with VoIP degradations and evaluate the system in terms of detection accuracy, robustness, latency, scalability, and deployment feasibility.

Keywords: Audio Deepfake Detection, Codec Compression, Machine Learning, Packet Loss, Real-Time Communication, Synthetic Speech, Voice over Internet Protocol

Table of Contents

List of Figures

List of Tables

List of Abbreviations

AAC	Advanced Audio Coding
AASIST	Anti-spoofing with Adaptive Softmax and Instance wise Temperature
ADD	Audio Deepfake Detection
ADC	Analog-to-Digital Conversion
AES	Advanced Encryption Standard
AGC	Automatic Gain Control
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
ASV	Automatic Speaker Verification
AUC	Area Under Curve (ROC metric)
CGNAT	Carrier-Grade Network Address Translation
CNN	Convolutional Neural Network
CM	Countermeasure (Spoofing Detection Model)
CQCC	Constant Q Cepstral Coefficients
DAC	Digital-to-Analog Converter
DL	Deep Learning
E2EE	End-to-End Encryption
ECDH	Elliptic Curve Diffie–Hellman
EER	Equal Error Rate
EVS	Enhanced Voice Services (audio codec)
FFT	Fast Fourier Transform
GAN	Generative Adversarial Network

GMM	Gaussian Mixture Model
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
ICE	Interactive Connectivity Establishment
IoT	Internet of Things
IP	Internet Protocol
ISP	Internet Service Provider
LCNN	Light Convolutional Neural Network
LFCC	Linear Frequency Cepstral Coefficients
LLM	Large Language Model
MFCC	Mel-Frequency Cepstral Coefficients
ML	Machine Learning
MQTT	Message Queuing Telemetry Transport
NAT	Network Address Translation
PCL	Phoneme-Guided Consistency Learning
PCM	Pulse Code Modulation
PLR	Packet Loss Rate
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic
RTC	Real-Time Communication
RTP	Real-time Transport Protocol
SDK	Software Development Kit
SFU	Selective Forwarding Unit
SIP	Session Initiation Protocol

SNR	Signal-to-Noise Ratio
SRTP	Secure Real-time Transport Protocol
SSL	Self-Supervised Learning
STFT	Short-Time Fourier Transform
STUN	Session Traversal Utilities for NAT
TCP	Transmission Control Protocol
TTS	Text-to-Speech
TURN	Traversal Using Relays around NAT
UDP	User Datagram Protocol
UI	User Interface
USB	Universal Serial Bus
VC	Voice Conversion
VAD	Voice Activity Detection
VoIP	Voice over Internet Protocol
WAV	Waveform Audio File Format
XGBoost	Extreme Gradient Boosting
XLSR	Cross-Lingual Speech Representation

1. INTRODUCTION

1.1 Background

Recent advances in Artificial Intelligence (AI), particularly in Text-to-Speech (TTS) and Voice Conversion (VC) technologies, have significantly improved the quality and realism of synthetic speech generation. The adoption of deep neural networks, self-supervised speech representations, and neural vocoders has enabled the synthesis of speech that closely resembles natural human speech in terms of intelligibility, speaker characteristics, and prosodic features.

Research in neural speech synthesis has evolved rapidly over the last decade. Early end-to-end architectures such as Tacotron [1] demonstrated the feasibility of generating natural-sounding speech directly from text. Subsequent developments introduced improvements in prosody modeling [2], style control [3], multi-speaker synthesis and voice transfer [4], further enhancing the realism and flexibility of generated speech. More recently, voice cloning systems have shown the ability to reproduce a target speaker's voice using only a limited amount of reference audio.

As synthetic speech quality continues to improve, distinguishing between human and AI-generated speech has become increasingly challenging. This has led to growing research interest in Audio Deepfake Detection (ADD), which focuses on identifying artifacts and inconsistencies introduced during the speech generation process. Existing ADD approaches utilize various signal processing, machine learning, and deep learning techniques to differentiate synthetic speech from authentic human speech. Public benchmark datasets and evaluation challenges have further accelerated research in this area, resulting in substantial improvements in detection performance under controlled experimental conditions.

1.2 Motivation

The increasing accessibility of AI-based speech synthesis and voice cloning technologies has lowered the barrier for generating highly convincing synthetic speech. Modern voice generation systems are widely available through open-source frameworks, cloud services, and commercial applications, making it possible to create realistic voice im-

personations with minimal technical expertise and limited reference audio.

The misuse of synthetic speech presents emerging challenges for cybersecurity, digital trust, and communication security. AI-generated voices can be used to impersonate individuals during phone calls, bypass voice-based verification mechanisms, conduct social engineering attacks, and spread misinformation through fabricated audio content. As voice communication continues to play an important role in online meetings, internet telephony, customer service systems, and voice authentication platforms, the potential impact of such attacks is increasing.

Although significant progress has been made in deepfake speech detection research, most existing solutions are evaluated using clean and carefully curated datasets that do not accurately represent real-world communication environments. Practical deployment requires detection systems that remain reliable under communication degradations and operational constraints encountered in everyday voice transmission systems. Therefore, there is a strong need to investigate detection approaches that can operate effectively in realistic communication scenarios while maintaining acceptable accuracy and latency.

1.3 Problem Definition

Existing Audio Deepfake Detection (ADD) systems have demonstrated strong performance on clean, offline speech datasets under controlled laboratory conditions. However, real-world communication systems primarily operate through Voice over Internet Protocol (VoIP) infrastructures, where transmitted audio is commonly affected by codec compression, packet loss, transmission artifacts, background noise, and latency constraints.

These communication degradations alter speech characteristics and may remove or distort features that detection models rely upon for identifying synthetic speech. Consequently, detection performance achieved in controlled research environments may not accurately reflect performance in practical deployment scenarios. Despite ongoing advances in deepfake detection research, there remains a gap between research-oriented detection models and deployment-ready systems capable of operating in real-time communication environments. Robust mechanisms that can reliably detect synthetic speech

while maintaining low-latency operation under degraded VoIP conditions remain limited. This project seeks to address this gap by investigating and evaluating real-time synthetic speech detection approaches in realistic VoIP communication scenarios.

1.4 Objectives

The objectives of this project are:

- To develop and validate an audio deepfake detection service using existing machine learning models for synthetic speech classification
- To integrate the developed detection service with a real-time communication (RTC) system and evaluate the effects of codec compression, packet loss, and background noise on detection performance and latency

1.5 Scope and Application

The project focuses on audio-based synthetic speech detection within VoIP communication environments under realistic transmission degradations. The system will process streaming audio segments and perform near real-time inference using machine learning and audio processing techniques.

1.5.1 Project Capabilities

The project is capable of the following:

- Detect whether incoming speech audio is real or AI-generated (synthetic speech) in near real-time
- Processes streaming audio chunks instead of requiring full audio files
- Operates under simulated VoIP transmission conditions for evaluation and testing
- Supports low-latency inference pipeline suitable for real-time communication scenarios
- Provide measurable detection confidence and performance metrics under different scenarios

1.5.2 Project Limitations

The scope of this project is subject to the following limitations:

- Performance may degrade when exposed to unseen or highly advanced neural codec-based synthetic speech models
- Limited generalization to languages or accents not well represented in training data (e.g., Nepali-specific datasets may be limited)
- Accuracy may reduce under extreme noise conditions or highly compressed audio streams
- Real-time performance depends on hardware capability and model complexity trade-offs
- Dataset availability constraints may limit coverage of diverse real-world VoIP scenarios and attack types

1.5.3 Project Applications

The potential applications of the proposed system includes:

- **Fraud and Social Engineering Detection:** Detects voice spoofing in phishing calls, impersonation scams, and financial fraud attempts
- **VoIP Communication Security:** Enhances security in internet-based calling systems (e.g., Zoom-like platforms, SIP-based VoIP systems) by flagging synthetic speech in real-time
- **Voice Authentication Systems:** Acts as a countermeasure layer for speaker verification systems used in banking, telecom, and identity verification services
- **Call Center Security:** Helps verify authenticity of customer or agent voices in customer support and outsourcing environments
- **Cybersecurity and Digital Forensics:** Assists in analyzing suspicious audio recordings for potential AI-generated manipulation
- **Online Meeting Integrity:** Supports detection of synthetic or manipulated voices during virtual meetings and remote collaboration tools
- **Media and Content Verification:** Helps verify authenticity of audio content used in news, interviews, and public broadcasts

2. LITERATURE REVIEW

The rapid evolution of Artificial Intelligence (AI) and deep learning has enabled the creation of highly realistic synthetic media, commonly known as "deepfakes" [5, 6]. While deepfake research initially focused on video, achieving high accuracy in detecting manipulated facial movements, audio deepfake detection has emerged as a critical and urgent challenge [5, 7]. These synthetic voices, which mimic human subtleties like tone, rhythm, and accent, are increasingly used for identity theft, misinformation, and financial fraud [5, 8]. Recent incidents have demonstrated the severity of the threat, such as scammers using AI-impersonated voices to obtain fraudulent money transfers worth hundreds of thousands of dollars [9, 10, 11]. Protecting Real-Time Communication (RTC) systems, such as VoIP and video conferencing, is particularly vital as these platforms have become cornerstones of identity verification in modern digital interactions [10, 12].

2.1 Evolution of Synthetic Speech and Voice Cloning

Audio deepfakes are generally categorised into two primary techniques: Text-to-Speech (TTS) and Voice Conversion (VC) [8, 13].

Speech synthesis, or TTS, converts textual input into speech that emulates a target speaker's voice [5, 8]. Traditional parametric and concatenative methods often produced "robotic" sounds, but modern deep learning has shifted the field toward neural vocoders and autoregressive models like Tacotron 2, DeepVoice, and FastSpeech, which generate highly natural-sounding speech [13, 14].

Voice conversion involves modifying the vocal characteristics of a source speaker to mimic a target speaker while preserving the original linguistic content [8, 13]. Modern VC systems often utilize End-to-End (E2E) architectures, employing neural vocoders like HiFi-GAN and flow-based models to achieve real-time, high-fidelity conversion [13, 15].

Voice cloning is the process of adapting a TTS system to unseen speakers, often categorized as few-shot or zero-shot cloning [13]. Zero-shot cloning is particularly dangerous as it requires only a short audio clip (as little as a few seconds) to generate speech with

the target's voice characteristics without needing to fine-tune the model [13, 12].

Current landscapes include Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and Diffusion Models [5, 13]. Recent models like VALL-E treat speech synthesis as a conditional language modelling task using Large Language Models (LLMs) and neural audio codecs to achieve human-parity synthesis [13, 12].

2.2 Rise of Audio Deepfake Detection

Detection systems typically consist of a front-end feature extractor and a back-end classifier [8, 14].

Early systems relied on handcrafted acoustic features:

- Mel-Frequency Cepstral Coefficients (MFCCs): Represent the short-term power spectrum and are effective for clean environments [5, 8].
- Linear Frequency Cepstral Coefficients (LFCCs): Retain more detailed high-frequency information, suitable for detecting spectral distortions in deepfakes [8, 16].
- Constant Q Cepstral Coefficients (CQCCs): A standard baseline in ASVspoof challenges used to capture anomalies in the time-frequency domain [8, 16].

Modern detectors leverage sophisticated neural architectures like Convolutional Neural Networks (CNNs) for capturing spectrogram artifacts, and Recurrent Neural Networks (RNNs/LSTMs) for detecting irregularities in speech rhythm and prosody [5, 8]. Models like AASIST use Graph Neural Networks (GNNs) to model complex spectro-temporal dependencies [8, 11].

SSL models pre-trained on massive unlabeled datasets have revolutionized detection. Models such as wav2vec 2.0, XLS-R, and WavLM generate high-quality contextual representations that excel in cross-lingual and out-of-domain scenarios [9, 8, 14].

Key datasets include the ASVspoof challenge series (2015–2024), providing standardized evaluations for synthetic speech [7, 12, 8]. Other datasets like In-the-Wild (ITW) and WaveFake introduce more diverse generative models and real-world conditions [8, 14].

2.3 Real-Time Communication (RTC) Systems

RTC systems introduce unique signal alterations that impact detection performance [10, 11]. The "black-box" processing pipeline of RTC platforms typically includes sender-side pre-processing (noise suppression, echo cancellation), encoding, network transmission (subject to jitter and packet loss), and receiver-side post-processing (gain control, bandwidth extension) [11].

Codecs such as Opus, SILK, EVS, and AMR-WB are used to compress digital audio [11, 16]. These significantly impact audio factors like Signal-to-Noise Ratio (SNR) and energy loss, often erasing subtle forgery cues [17, 16]. Network-induced data loss and timing variations degrade the performance of feature-based systems. Packet Loss Rates (PLR) in wireless channels can introduce distorted transmissions, reducing the robustness of deepfake detectors [16].

2.4 Impact of RTC Distortions on Deepfake Detection

Existing models trained on clean audio often fail to generalize to degraded VoIP environments [10, 16]. Compression results in the loss of high-frequency information, which is critical for features like LFCC [8, 16]. Research shows that models trained on static recordings suffer significant performance declines when challenged with codec-distorted audio [10, 16].

The combined effects of compression and network-induced packet loss significantly reduce speech quality [16]. Benchmarking has shown that models like GMM-CQCC and LCNN-LFCC exhibit systematic declines in robustness as PLR increases [16]. Variation in processing logic across different platforms (e.g., Zoom vs. WeChat) leads to significantly different distortion distributions, posing a major challenge for model generalization [11].

2.5 Recent Research on RTC-Aware Deepfake Detection

Recent efforts include the RTCFake dataset, which captures real-world black-box distortions from mainstream platforms [11], and the ADD-C test dataset, designed to evaluate robustness under multiple codecs and PLRs [16]. The Phoneme-Guided Consistency Learning (PCL) strategy has been proposed to learn platform-invariant features [11].

Current systems often lack analysis for low-latency deployment on edge devices like smartphones [17]. Furthermore, models trained exclusively on online or offline data suffer from severe domain mismatch [11, 10].

2.6 Research Gap Analysis

Despite progress, critical gaps remain:

1. Systematic RTC Impact: Limited research exists on the combined impact of codec compression, packet loss, and noise on both detection performance and latency [16, 17].

2. Resource Constraints: There is a lack of lightweight models optimized for real-time inference in resource-constrained RTC environments [17, 15].

3. Real-World Transmission: Most datasets rely on simulated distortions rather than real-world black-box transmission [11].

Table 2-1: Summary of Literature Review

Title	**Methodology**	**Key Inferences**
A Survey on Speech Deepfake Detection (2024) [8]	Systematic analysis of over 200 publications (up to March 2024) covering the entire detection pipeline; Evaluates both fully synthetic and partially manipulated audio	Shift from handcrafted features toward deep embeddings from SSL models like wav2vec 2.0; OC-Softmax as a superior loss function

Towards the Development of a Real-Time Deepfake Audio Detection System in Communication Platforms [10]	Evaluates models trained on ASVspoof 2019 under live Microsoft Teams communication streams. Assesses deployment performance using precision, recall, and F-score metrics.	Models trained on offline datasets degrade significantly in real communication environments due to compression and platform artifacts.
RTCFake: Speech Deepfake Detection in Real-Time Communication [11]	Constructs a 600-hour dataset through real-world black-box transmission across seven communication platforms including Zoom and WeChat. Proposes **Phoneme-Guided Consistency Learning (PCL)** to learn platform-invariant speech representations.	RTC transmission introduces severe domain mismatch not captured by traditional datasets such as ASVspoof. Phoneme-level representations remain more stable than frame-level acoustic features under platform-induced distortions, improving generalization across unseen communication platforms.
ASVspoof 5: Crowdsourced Speech Data, Deepfakes, and Adversarial Attacks at Scale [12]	Introduces crowdsourced speech recordings, neural audio codecs, and adversarial attacks targeting both deepfake detection and speaker verification systems.	Neural codecs and adversarial perturbations significantly reduce detector reliability; robust calibration is required.

Real-Time Detection of AI-Generated Speech for Deepfake Voice Conversion [15]	Introduces the DEEP-VOICE dataset and evaluates lightweight machine learning algorithms including Random Forest and XGBoost for short-window real-time inference.	XGBoost achieves high detection performance with extremely low inference latency, demonstrating that optimized traditional machine learning approaches remain viable for edge and real-time deployment scenarios.
Benchmarking Audio Deepfake Detection Robustness in Real-World Communication Scenarios [16]	Introduces the **ADD-C benchmark**, simulating six communication codecs and packet loss rates ranging from 0% to 20%.	Models such as LCNN and AASIST experience performance decreases with increasing packet loss; codec-aware augmentation improves robustness.
MultiAPI Spoof: A Multi-API Dataset and Local-Attention Network for Speech Anti-Spoofing Detection [18]	Constructs a dataset containing spoofed speech generated from 30 different APIs and proposes a Local-Attention enhanced Nes2Net architecture for fine-grained feature learning.	Source diversity significantly affects detector generalization. Local-attention mechanisms improve robustness across different synthesis systems, while API tracing introduces attribution as an emerging research direction.

3. REQUIREMENT ANALYSIS

3.1 Software Requirements

The proposed Audio Deepfake Detection (ADD) framework consists of multiple interacting components, including RTC communication infrastructure, audio processing modules, machine learning models, deployment services, and system integration layers. Consequently, a diverse software ecosystem is required to support development, experimentation, deployment, and evaluation. The software requirements are categorized according to their functional roles within the system architecture.

3.1.1 Operating System and Development Environment

1. **Operating System (Linux, Windows, or macOS)**

   The proposed system can be developed and executed on modern operating systems including Linux, Windows, and macOS. Linux-based environments are particularly suitable for machine learning training, server deployment, and containerized applications due to their stability, performance, and compatibility with GPU acceleration frameworks.

2. **Integrated Development Environment (IDE)**

   An Integrated Development Environment is required for software development, debugging, testing, and project management. Visual Studio Code is recommended due to its extensive support for Python, Go, Docker, and version control integration. The IDE facilitates efficient development and maintenance of both RTC and machine learning components.

3. **Version Control System (Git)**

   Git is used for source code management and version tracking throughout the software development lifecycle. It enables experiment reproducibility, collaborative development, code backup, and systematic management of project changes.

3.1.2 Programming Languages

1. **Python 3.x**

Python serves as the primary programming language for implementing the Audio Deepfake Detection service. It provides extensive support for machine learning, audio processing, data analysis, and model deployment. Its rich ecosystem of scientific computing libraries makes it particularly suitable for research-oriented development and rapid prototyping.

2. **Go Programming Language**

   Go serves for implementing RTC-related services, media handling components, signaling servers, RTP stream processing, and integration middleware. Its lightweight concurrency model, efficient networking capabilities, and low runtime overhead make it well-suited for real-time communication systems that process multiple concurrent audio streams.

3.1.3 RTC and Communication Infrastructure

1. **WebRTC and RTP-Based Communication Libraries**

   RTC communication requires software libraries capable of handling audio streaming, RTP packet processing, session establishment, and media transport. These technologies provide the foundation for simulating realistic VoIP and RTC environments and enable integration between communication systems and the proposed detection service.

2. **Media Server Infrastructure**

   Media server components may be utilized to manage audio streams in multi-user RTC environments. Such systems facilitate stream routing, forwarding, session management, and centralized audio access. They provide a practical integration point for deploying the proposed detection framework within real-world communication systems.

3. **Streaming and API Frameworks (FastAPI or gRPC)**

   The ADD system operates as an external service and therefore requires a communication interface for exchanging audio data and inference results. FastAPI provides a quick HTTP-based integration mechanism suitable for rapid development and prototyping. For low-latency streaming environments, gRPC offers

efficient binary communication and bidirectional streaming support, making it more appropriate for production-grade RTC deployments.

3.1.4 Machine Learning and Deep Learning Frameworks

1. **PyTorch**

   PyTorch is the primary deep learning framework used for implementing, training, and evaluating the proposed audio deepfake detection models. It provides automatic differentiation, GPU acceleration, flexible model development, and extensive support for state-of-the-art neural architectures.

2. **Hugging Face Transformers**

   The Hugging Face Transformers library provides access to pretrained self-supervised speech models such as WavLM, HuBERT, and Wav2Vec 2.0. These models serve as feature extraction backbones within the proposed framework and significantly reduce the computational cost associated with training large speech representation models from scratch.

3. **Scikit-learn**

   Scikit-learn is used for data preprocessing, statistical analysis, and performance evaluation. It provides implementations of commonly used evaluation metrics such as accuracy, precision, recall, F1-score, ROC curves, and confusion matrices that are essential for assessing detection performance.

3.1.5 Audio Processing Libraries

1. **Librosa**

   Librosa provides functionality for loading audio data, feature extraction, spectrogram generation, resampling, and signal analysis. It is extensively used during dataset preparation and experimental evaluation.

2. **SoundFile**

   SoundFile supports efficient reading and writing of audio files in multiple formats and serves as a reliable backend for audio data management.

3. **NumPy and SciPy**

   NumPy and SciPy provide numerical computing and signal processing capabilities required for audio preprocessing, filtering, statistical analysis, and implementation of custom signal-processing algorithms.

3.1.6 Data Storage and System Services

1. **PostgreSQL**

   PostgreSQL may be used to store system metadata, inference logs, experimental results, user activity records, and configuration information. Its reliability and support for structured data management make it suitable for long-term storage requirements.

2. **Redis**

   Redis may be employed as an in-memory data store and message broker for real-time communication between system components. It can be used for temporary buffering, caching recent predictions, managing task queues, and facilitating asynchronous processing in low-latency environments.

3.1.7 Deployment and Containerization Tools

1. **Docker**

   Docker is used to containerize the individual components of the proposed framework, including the ADD service, API server, database services, and RTC middleware. Containerization simplifies deployment, improves portability, and ensures consistent execution across different development and production environments.

2. **Docker Compose**

   Docker Compose may be used to orchestrate multiple interconnected services such as the detection engine, API layer, Redis instance, and PostgreSQL database. It simplifies system configuration and supports rapid deployment of the complete architecture.

3.2 Hardware Requirements

The proposed Audio Deepfake Detection (ADD) system involves real-time audio processing, deep learning-based inference, and simulation of RTC/VoIP communication conditions. Therefore, appropriate hardware resources are required to support data acquisition, model training, real-time inference, and system evaluation. The hardware requirements are categorized based on their functional roles within the system.

1. **Processor (Multi-core CPU)**

   A multi-core CPU is required to handle concurrent system operations such as audio preprocessing, stream ingestion, API handling, and RTC simulation tasks. Since the proposed system processes continuous audio streams in real time, parallel execution capability is essential. A modern processor with at least 4 to 8 cores is recommended to ensure smooth execution of both inference and communication-related tasks.

2. **Main Memory**

   Sufficient memory is required for loading audio data, managing streaming buffers, and executing machine learning models. During real-time processing, multiple audio chunks may be stored temporarily for windowing and aggregation, which increases memory demand. A minimum of 8 GB RAM is required for basic execution; however, 16 GB or higher is recommended for stable training and multitasking performance.

3. **Graphics Processing Unit (GPU with CUDA Support)**

   A dedicated GPU is highly recommended for training and evaluating deep learning models used in audio deepfake detection. Models such as WavLM, Wav2Vec 2.0, and transformer-based classifiers involve computationally intensive matrix operations that benefit significantly from GPU acceleration. CUDA-enabled NVIDIA GPUs are preferred, as PyTorch and related frameworks provide optimized support for GPU-based computation, significantly reducing training time.

4. **Storage Device**

A solid-state drive (SSD) is recommended for efficient storage and retrieval of large audio datasets, pretrained models, and experimental logs. Audio datasets and deep learning checkpoints can occupy significant disk space, and SSDs provide faster read/write speeds compared to traditional hard drives, improving overall system performance during training and evaluation.

5. **Audio Input Device (Microphone)**

   A microphone is required for capturing real-time speech samples during testing and system evaluation. It is used to simulate real-world RTC conditions where speech is acquired through user input devices. The quality of the microphone may influence signal clarity, but the system is designed to be robust against moderate recording variations.

6. **Audio Output Device (Headphones or Speakers)**

   Audio playback devices are required for monitoring reconstructed speech during RTC simulation and verification of system behavior. Headphones are preferred for reducing external noise interference during evaluation and testing phases.

7. **Network Connectivity**

   A stable internet connection is required to simulate real-world RTC environments involving packet transmission, latency, jitter, and packet loss conditions. Since the proposed system evaluates performance under VoIP-like conditions, network stability plays a key role in ensuring realistic testing scenarios.

8. **High-Performance Computing (HPC) / Cloud GPU Support**

   For large-scale training or experimentation with deep self-supervised models, cloud-based GPU resources (such as NVIDIA T4, V100, or A100 instances) may be utilized. This is particularly useful when training is computationally expensive or when local hardware resources are insufficient.

4. SYSTEM ARCHITECTURE AND METHODOLOGY

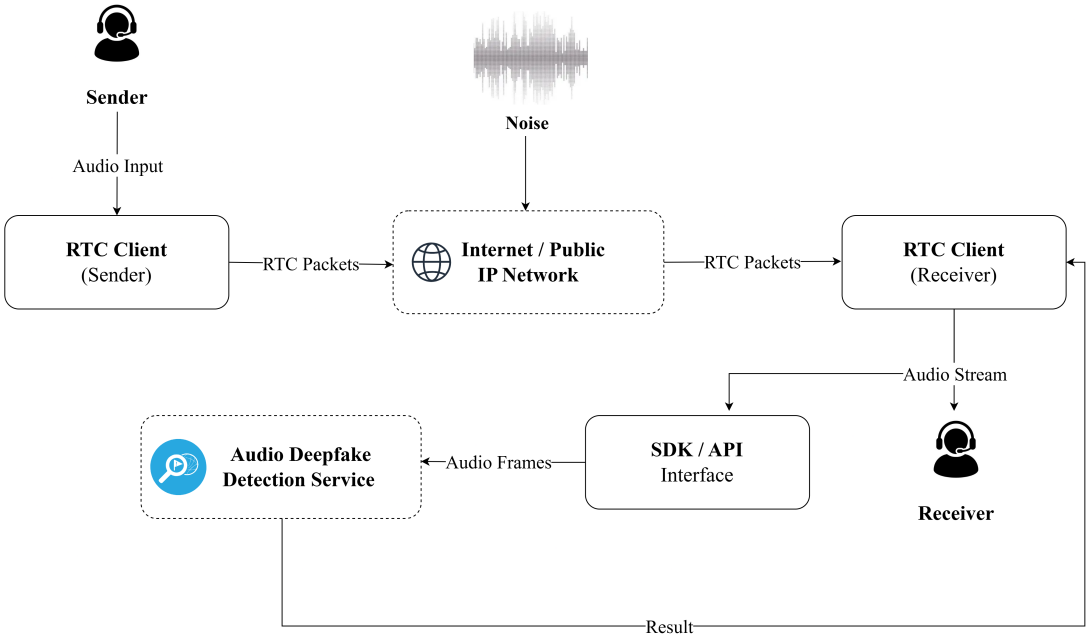


Figure 4-1: High-Level Design of the Proposed System

Figure 4-1 presents the overall architecture of the proposed RTC-aware Audio Deepfake Detection (ADD) framework. The system is designed to detect synthetic speech during live communication sessions by operating on audio streams that have undergone realistic real-time communication (RTC) processing and transmission.

Unlike conventional deepfake detection approaches that operate on clean offline recordings, the proposed framework considers the complete communication pipeline through which speech is transmitted between users. During an RTC session, audio is captured at the sender side, compressed using communication codecs, packetized for network transport, transmitted across IP networks, reconstructed at the receiver side, and finally delivered for playback. Throughout this process, the speech signal experiences multiple transformations including codec compression, packet loss recovery, jitter buffering, and audio enhancement. These operations may alter or suppress artifacts commonly exploited by deepfake detection systems. Therefore, the proposed framework performs detection on the final reconstructed speech signal observed at the receiver side, closely

reflecting practical deployment conditions.

The architecture consists of four major subsystems:

1. **Sender Side (RTC Client):** Captures speech from the speaker, performs audio preprocessing, codec encoding, and packetization before transmission.
2. **Network Transmission Layer:** Transfers audio packets across IP-based networks where latency, jitter, packet loss, and routing variability may affect signal integrity.
3. **Receiver Side (RTC Client):** Reconstructs the transmitted audio stream through packet reordering, jitter buffering, decoding, packet loss concealment, and audio enhancement operations.
4. **Audio Deepfake Detection Service (ADD):** Operates as an external inference service that receives reconstructed audio from the receiver pipeline, extracts robust speech representations, performs deepfake classification, and generates spoofing risk assessments.

The proposed system operates within a standard RTC communication scenario involving two communicating clients. When a call is established, speech captured by the sender-side RTC client is processed through the conventional transmission pipeline, including audio acquisition, preprocessing, encoding, packetization, encryption, and network transport. The transmitted packets traverse the communication network and are received by the receiver-side RTC client. The receiver reconstructs the speech signal through packet reordering, jitter buffering, packet loss concealment, decoding, and audio playback preparation.

Rather than modifying the communication process itself, the proposed framework introduces an Audio Deepfake Detection (ADD) service at the receiver side of the communication pipeline. After the speech signal has been reconstructed, a copy of the received audio stream is forwarded to the detection service for analysis. The ADD service extracts robust speech representations from the reconstructed audio and performs inference using a deep learning-based detection model. Based on the extracted features, the system generates a confidence score and a classification decision indicating whether the observed speech is real or synthetically generated.

The detection service is integrated through a non-intrusive plugin architecture, allowing it to operate independently from the RTC communication stack. A key design principle of the proposed framework is the separation between communication infrastructure and detection logic. Rather than modifying the underlying RTC protocols or media transport mechanisms, the system performs analysis on the reconstructed receiver-side audio stream. This approach preserves compatibility with existing RTC platforms while enabling the detection model to operate on speech signals that have already undergone realistic transmission, compression, buffering, and reconstruction processes.

Developing the sender-side and receiver-side communication pipeline (RTC system) is important because audio transmitted through RTC systems undergoes multiple transformations before reaching the receiving user. These transformations introduce codec artifacts, quantization effects, packet loss recovery artifacts, buffering effects, and other network-induced distortions that may affect the discriminative characteristics used for deepfake detection. Consequently, the proposed system evaluates detection performance under realistic communication constraints rather than on pristine studio-quality recordings, thereby providing a more representative assessment of deployment-time performance.

The following sections describe each subsystem in detail, including the sender-side transmission pipeline, network transmission layer, receiver-side reconstruction process, and the proposed Audio Deepfake Detection service.

4.1 Sender Side (RTC Transmission Pipeline)

The sender-side is responsible for transforming a speaker's voice into a compressed network stream suitable for real-time communication. In the proposed framework, the sender-side pipeline is treated as a standard RTC subsystem and is not modified by the detection mechanism and serves as a realistic source of audio data that reflects practical deployment conditions found in VoIP and conferencing systems.

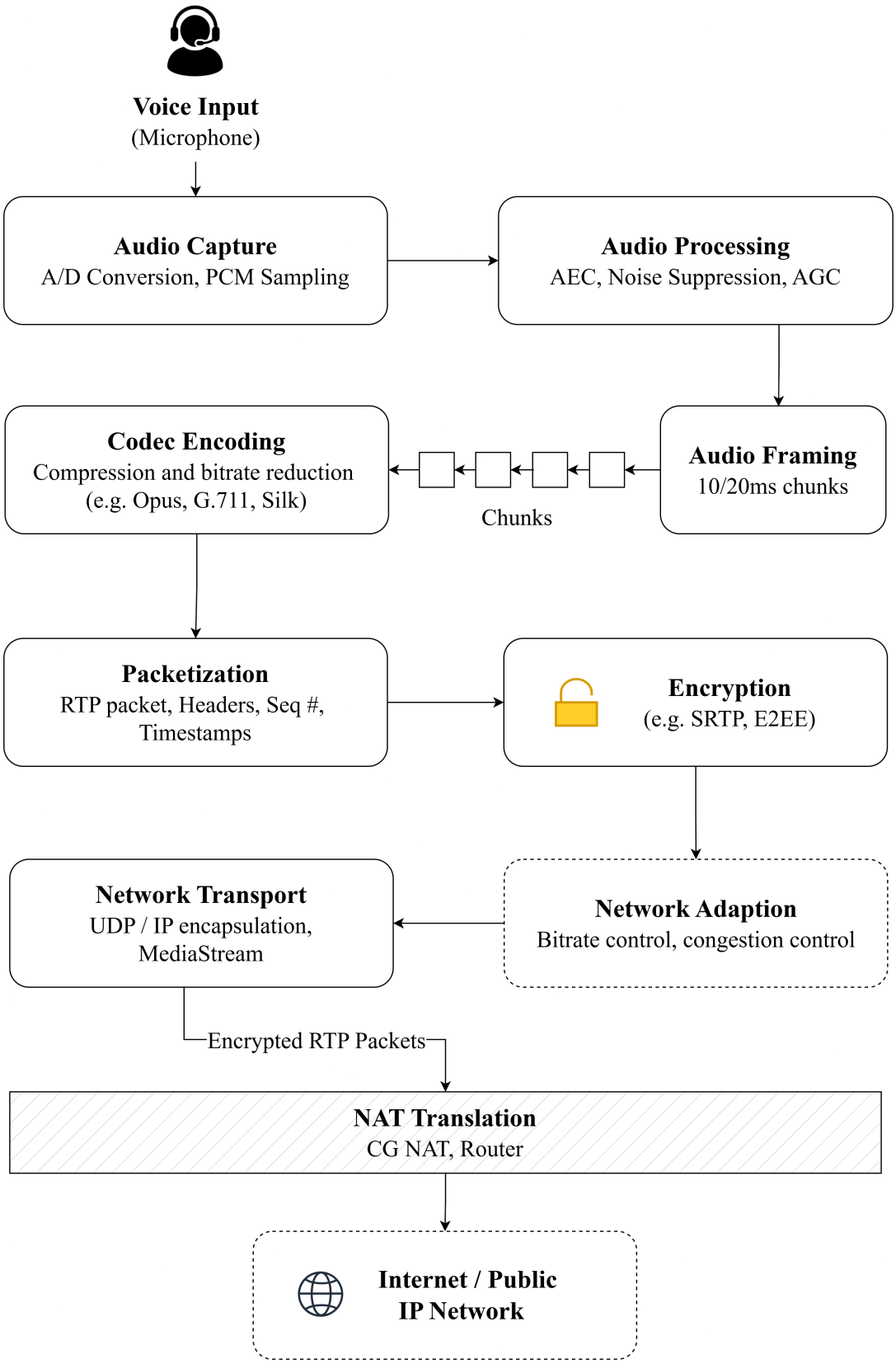


Figure 4-2: Sender-side RTC Transmission Pipeline

4.1.1 Audio Capture and Pre-processing

The transmission process begins with acquisition of speech through a microphone device. The analog speech waveform is converted into a digital signal through Analog-to-Digital Conversion (ADC), where sampling and quantization are performed at a predefined sampling rate (typically ranging from 16 kHz to 48 kHz).

Let the continuous-time speech signal be represented as $x(t)$ which is transformed into a discrete-time representation $x[n]$ through the sampling process.

Depending on the operating system and device capabilities, additional preprocessing may be applied, including acoustic echo cancellation, noise suppression, and gain normalization. These operations improve speech intelligibility but may also alter spectral characteristics relevant to deepfake analysis.

Here, the signal remains in a near-natural form but may already include environmental noise and recording artifacts. This stage introduces uncontrolled variability (speaker device, environment), which contributes to domain diversity in downstream detection.

4.1.2 Audio Framing

The processed audio stream is divided into small fixed duration frames, typically ranging from 10ms to 20ms. Framing allows the system to process and transmit audio incrementally rather than waiting for larger audio segments.

Each frame serves as an independent unit for encoding and network.

4.1.3 Encoding and Compression

The preprocessed audio signal is compressed using a speech codec before transmission. Modern RTC systems typically employ codecs such as Opus due to their low latency and adaptive bitrate capabilities.

Let $x[n]$ denote the input waveform and $C(\cdot)$ denote the codec transformation:

$$x_c = C(x[n]) \tag{4.1}$$

where x_c is the compressed bitstream representation.

Compression reduces transmission bandwidth requirements but introduces codec-specific distortions, including quantization noise, spectral smoothing, and loss of fine-grained acoustic information. These effects may partially mask or modify artifacts generated by synthetic speech systems.

4.1.4 Packetization

The compressed audio stream is segmented into small packets suitable for real-time delivery. Each packet is encapsulated into transport packets containing timing and synchronization metadata.

The packetization process is represented as

$$x_c \rightarrow \{p_1, p_2, \ldots, p_n\} \tag{4.2}$$

where p_i represents an individual network packet.

Each packet generally contains a compressed payload along with sequence numbers and timestamps required for stream reconstruction at the receiver.

4.1.5 Encryption and Security

To ensure confidentiality and protect user privacy, voice packets are encrypted before transmission. The proposed system employs end-to-end encryption (E2EE), ensuring that only the communicating participants can access the contents of the conversation. Even if network traffic is intercepted, unauthorized parties cannot recover the original audio data without the corresponding cryptographic keys.

During call establishment, a secure key exchange mechanism is performed between the communicating parties. Modern public-key cryptographic algorithms such as X25519 (Elliptic Curve Diffie–Hellman) are used to derive a shared session key without transmitting the key itself over the network.

Once a secure session key has been established, the audio stream is encrypted using a symmetric encryption algorithm such as AES-256-GCM or ChaCha20-Poly1305. These algorithms provide both confidentiality and integrity protection for transmitted audio packets.

The encrypted audio packets are then encapsulated using the Secure Real-Time Transport Protocol (SRTP), which provides secure delivery of real-time voice data. Finally, the packets are transmitted over UDP, a low-latency transport protocol commonly used in real-time communication systems due to its minimal transmission overhead and reduced delay characteristics.

4.1.6 Network Transport

Following encryption, the protected audio packets are prepared for network transmission. The encrypted SRTP packet is encapsulated within a UDP datagram, which provides a lightweight and low-latency transport mechanism suitable for real-time communication.

The UDP datagram is subsequently encapsulated within an IP packet containing the source and destination addressing information required for routing across the network. This layered encapsulation enables intermediate networking devices to forward packets toward the intended recipient.

Once packet construction is complete, the operating system transmits the packet through the available network interface, such as Wi-Fi or cellular data. The sender continuously generates, encrypts, encapsulates, and transmits packets throughout the duration of the call, forming a secure real-time voice stream between communicating users.

4.2 Network Transmission Layer

The network transmission layer represents the communication channel between the sender and receiver in an RTC system. Its primary function is to transport encoded audio packets across IP-based networks while satisfying the low-latency requirements of real-time communication. Unlike controlled laboratory conditions, practical communication networks introduce stochastic impairments that can significantly affect speech signal quality.

In the proposed framework, the network layer is treated as a black-box transmission channel that introduces realistic communication distortions. This allows the evaluation of the deepfake detection system under conditions representative of real-world VoIP and RTC deployments.

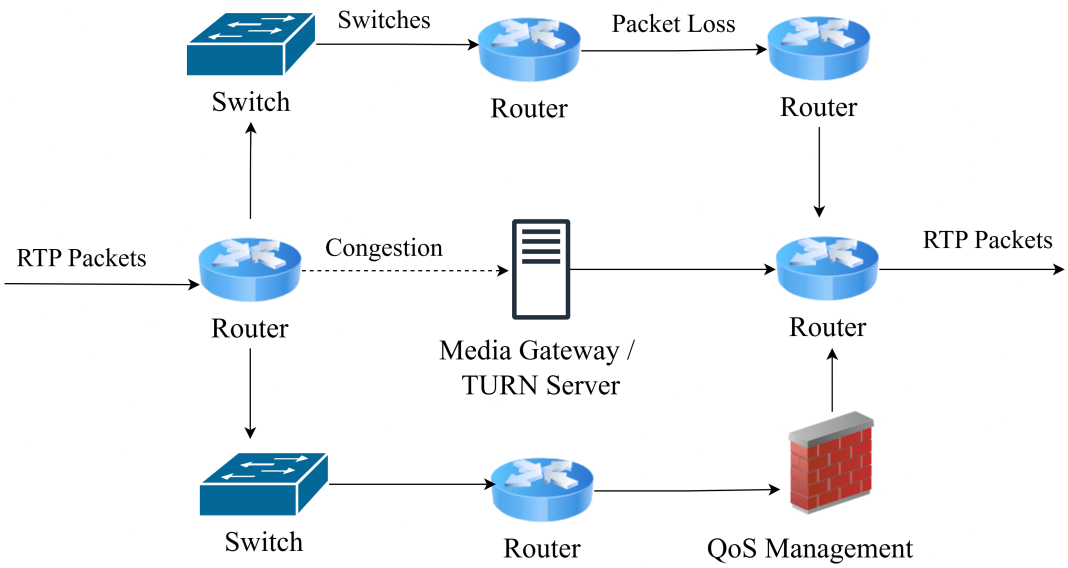


Figure 4-3: Network Impairments

4.2.1 Packet Routing in IP Networks

After packetization and encryption, audio packets are transmitted through packet-switched IP networks using transport protocols such as UDP, commonly combined with the Real-time Transport Protocol (RTP).

Packets traverse multiple intermediate routing nodes before reaching the receiver. Due to dynamic routing decisions and varying network conditions, packet delivery is not guaranteed to preserve timing consistency and may exhibit variable delays.

The transmission process can be represented as

$$p_i \rightarrow R_1 \rightarrow R_2 \rightarrow \cdots \rightarrow R_k \rightarrow p_i' \tag{4.3}$$

where, R_k denotes intermediate routing nodes and p_i' represents the received version of packet p_i.

Consequently, packet delivery in RTC systems is inherently probabilistic and subject to network-dependent variations.

4.2.2 Latency, Jitter, and Packet Loss

Three primary network impairments affect real-time audio communication:

Latency: The end-to-end delay between packet transmission and reception. Latency arises from propagation delay, routing overhead, congestion, and buffering operations.

Jitter: Variation in packet arrival times caused by fluctuations in network conditions. Excessive jitter can disrupt continuous audio playback and requires compensation mechanisms at the receiver.

Packet Loss: Failure of packets to reach the destination due to congestion, buffer overflow, transmission errors, or wireless link instability.

As a result, the received packet sequence may differ from the transmitted sequence:

$$p_1, p_2, \ldots, p_n \rightarrow p'_1, p'_2, \ldots, p'_m \tag{4.4}$$

where $m \leq n$, indicating that some transmitted packets may be delayed, reordered, or lost during transmission.

4.2.3 RTC Channel Characteristics

In practical RTC environments, network impairments interact with codec behavior and adaptive communication mechanisms. Modern RTC systems dynamically adjust transmission parameters in response to changing network conditions.

Common channel characteristics include:

- Time-varying packet loss rates
- Delay fluctuations caused by congestion
- Burst packet losses in wireless networks
- Adaptive bitrate adjustment by the encoder
- Variable packet inter-arrival intervals

Consequently, the effective communication channel cannot be modeled as a simple additive noise process. Instead, it behaves as a complex stochastic system that modifies

both the temporal and spectral characteristics of transmitted speech.

4.2.4 Impact on Speech Signal Integrity

The combined effects of codec compression and network impairments reduce the fidelity of the received speech signal.

Major consequences include:

- Loss of fine-grained temporal information
- Discontinuities caused by missing packets
- Distortion of spectral characteristics
- Irregular playback due to high jitter

To maintain acceptable voie quality, real-time communication systems must be designed to tolerate these impairments through techniques such as adaptive encoding, buffering, packet sequencing, and error concealment.

The overall transformation from transmitted speech to received speech can be represented as

$$x_r(t) = T_{net}\left(C(x(t))\right) \tag{4.5}$$

where

- $C(\cdot)$ denotes the codec compression process,
- $T_{net}(\cdot)$ represents the network transmission effects,
- $x_r(t)$ is the received speech signal.

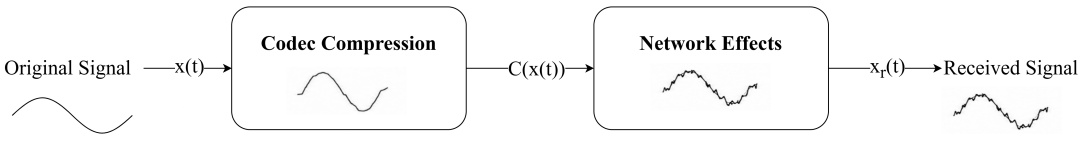


Figure 4-4: Codec and Network Effect on Speech Signal

These distortions directly influence the acoustic features for the downstream deepfake detection module and therefore must be considered when evaluating the RTC system.

4.3 Receiver Side (RTC Reconstruction Pipeline)

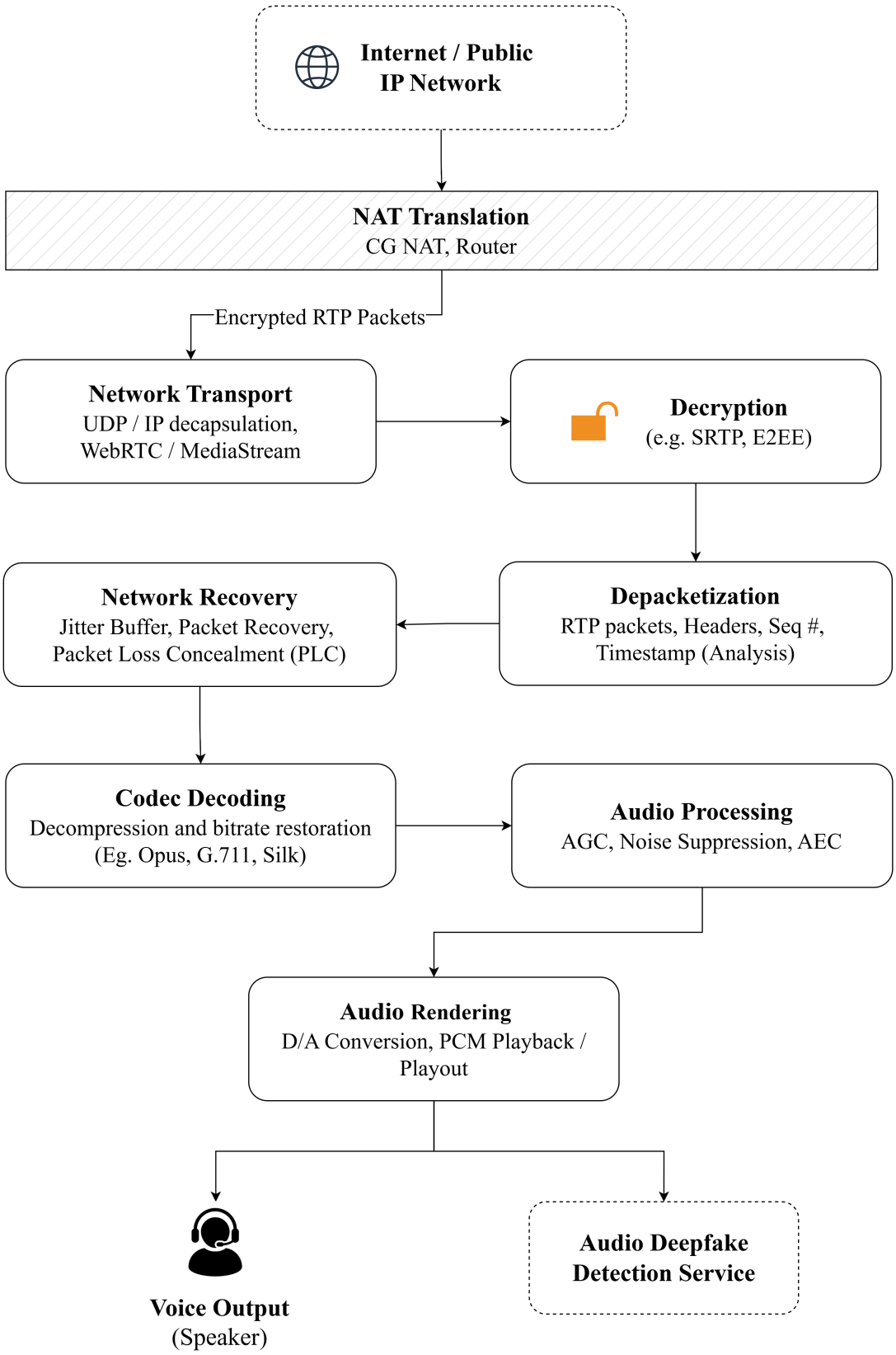


Figure 4-5: Receiver-side RTC Reconstruction Pipeline

The receiver side represents the reconstruction stage of the RTC system, where incoming network packets are converted back into a continuous audio stream for playback or downstream processing. This stage is critical because it produces the actual signal observed by the proposed deepfake detection service. In practical RTC deployments, the received signal is not a clean reconstruction of the original waveform; rather, it is a transformed version shaped by network impairments, packet handling mechanisms, codec decoding, and receiver-side enhancement operations.

In the proposed framework, the receiver pipeline is treated as a realistic reconstruction module that reflects the behavior of deployed VoIP and RTC systems. The output of this stage defines the input domain for the detection service, and therefore its distortions must be modeled carefully.

4.3.1 Packet Reception and Ordering

At the receiver, incoming audio packets are collected from the transport layer and processed in the order required for stream reconstruction. Due to network variability, packets may arrive out of order, delayed, duplicated, or missing.

Let the transmitted packet sequence be represented as

$$P_t = \{p_1, p_2, \ldots, p_n\}, \tag{4.6}$$

and let the set of packets successfully received at the destination be

$$P_r = \{p_1^{(r)}, p_2^{(r)}, \ldots, p_m^{(r)}\}, \quad m \leq n. \tag{4.7}$$

Here, $p_i^{(r)}$ denotes the received version of packet p_i. Sequence numbers and timestamps are used to detect packet loss and restore the correct temporal ordering of the stream.

The packet reception stage therefore performs the following operations:

- collection of arriving packets from the network stack,
- detection of missing or delayed packets using sequence identifiers,
- reordering of packets according to timestamps,

- preparation of packets for buffering and decoding.

Perfect reconstruction is not guaranteed when packet loss, delay variation, or reordering occurs.

4.3.2 Packet Authentication and Decryption

Received packets are first subjected to integrity verification to ensure that they have not been modified during transmission. Following successful verification, the encrypted payload is decrypted using the session keys established during call setup.

This process restores the original compressed audio data while maintaining confidentiality and protection against unauthorized access.

4.3.3 Depacketization

After decryption, the Real-Time Transport Protocol (RTP) headers are analyzed. Sequence numbers and timestamps contained within the packet headers provide information about packet ordering and playback timing.

These metadata fields enable the receiver to identify missing packets, detect out-of-order arrivals, and reconstruct the original audio sequence.

4.3.4 Jitter Buffer and Stream Stabilization

Since packets may arrive with irregular timing, RTC systems typically employ a jitter buffer to stabilize playback. The jitter buffer temporarily stores incoming packets and releases them at a controlled rate so that the downstream decoder receives a more regular stream.

The jitter buffer serves three main purposes:

- compensating for variation in packet arrival times,
- reordering packets before decoding,
- maintaining continuous audio playback with minimal interruptions.

However, this stabilization comes at the cost of additional delay and possible temporal modification of the signal. When network jitter is severe, the buffer may drop late

packets or duplicate previous frames to preserve continuity. This causes buffer output to temporally smoothed but not signal-faithful. This behavior is particularly important in deepfake detection because fine-grained temporal artifacts may be attenuated or altered by buffering operations.

4.3.5 Packet Loss Concealment

When packets are missing or arrive too late for playout, the receiver applies packet loss concealment (PLC) to generate a plausible replacement for the missing audio frame. PLC is commonly implemented using waveform repetition, interpolation, predictive synthesis, or codec-specific concealment strategies.

Let the buffered packet stream be represented as P_b. The concealment process can be expressed as $P_b \rightarrow P_c$, where P_c denotes the packet stream after missing frames have been concealed.

Although PLC improves perceptual continuity, it also introduces synthetic artifacts and may smooth over abrupt acoustic transitions. These modifications can partially mask the characteristics that are relevant for deepfake detection. PLC does not recover the original missing content; it only generates a perceptually acceptable approximation.

4.3.6 Audio Decoding Process

After buffering and concealment, the compressed payloads are passed to the codec decoder to reconstruct waveform samples. The decoder transformation can be modeled as

$$\hat{x}[n] = D(x_c), \tag{4.8}$$

where x_c denotes the compressed bitstream and $\hat{x}[n]$ represents the reconstructed discrete-time speech signal.

The characteristics of the reconstructed signal depend on the codec employed during transmission:

- **Opus:** The most widely used codec in modern RTC systems. Opus combines

linear prediction and transform-based coding techniques with adaptive bitrate control. During decoding, quantization artifacts, spectral smoothing, bandwidth limitations, and temporal distortions may be introduced, particularly under low-bitrate operating conditions.

- **G.711:** A waveform-based codec commonly used in traditional telephony systems. Decoding primarily involves inverse companding, resulting in relatively low algorithmic distortion but offering limited bandwidth efficiency compared to modern codecs.
- **AAC:** A transform-domain codec that employs perceptual audio coding techniques. Reconstruction may introduce quantization noise and perceptual smoothing effects, particularly in high-frequency regions of the spectrum.

In general, decoding is not a lossless process. Even under ideal network conditions, codec reconstruction introduces approximation errors due to compression and quantization. Consequently, the reconstructed signal preserves speech intelligibility while potentially altering fine-grained acoustic characteristics that may be relevant for deepfake detection.

4.3.7 Receiver-Side Audio Enhancement

After decoding, RTC systems often apply additional enhancement operations to improve perceptual quality. These operations are implementation-dependent but typically include automatic gain control, noise suppression, and acoustic echo cancellation.

Let the decoded signal be $\hat{x}[n]$. The receiver-side enhancement pipeline can be represented as

$$x_{rtc}[n] = T_{rtc}(\hat{x}[n]), \tag{4.9}$$

where $T_{rtc}(\cdot)$ denotes the combined receiver-side processing chain and $x_{rtc}[n]$ is the final reconstructed RTC audio signal.

Common enhancement modules include:

- **Automatic Gain Control (AGC):** normalizes signal amplitude to maintain consistent loudness,

- **Noise Suppression:** reduces stationary and non-stationary background noise,
- **Echo Cancellation:** suppresses feedback caused by speaker-to-microphone acoustic coupling.

These modules improve usability in real-time communication but can also alter amplitude dynamics, harmonic structure, and low-energy components of the speech signal. Such transformations are relevant because deepfake artifacts may be partially suppressed or distorted by receiver-side enhancement.

4.3.8 Output Signal for Deepfake Detection

The final output of the receiver pipeline is the audio stream presented to the proposed deepfake detection service. This signal represents the actual deployment-domain input rather than pristine source audio.

The overall receiver-side transformation may be summarized as

$$x_{out}[n] = \mathcal{T}_{rtc}(\mathcal{T}_{net}(C(x[n]))), \tag{4.10}$$

where

- $x[n]$ is the input audio waveform at the sender side,
- $C(\cdot)$ denotes codec compression and reconstruction,
- $\mathcal{T}_{net}(\cdot)$ denotes network-induced distortions,
- $\mathcal{T}_{rtc}(\cdot)$ denotes receiver-side buffering, concealment, and enhancement operations,
- $x_{out}[n]$ is the final reconstructed signal used for analysis.

The output audio is therefore temporally stabilized, perceptually enhanced, and potentially distorted by multiple stages of processing. This makes receiver-side modeling essential for evaluating deepfake detection under realistic RTC conditions.

4.4 Signaling and Call Session Management

The signaling and call session management subsystem is responsible for establishing, maintaining, and terminating communication sessions between users. Unlike the media transmission subsystem, signaling does not carry voice data. Instead, it coordinates call

setup procedures, user discovery, session negotiation, and call control operations.

This subsystem ensures that both participants are available, authenticated, and prepared before real-time voice transmission begins.

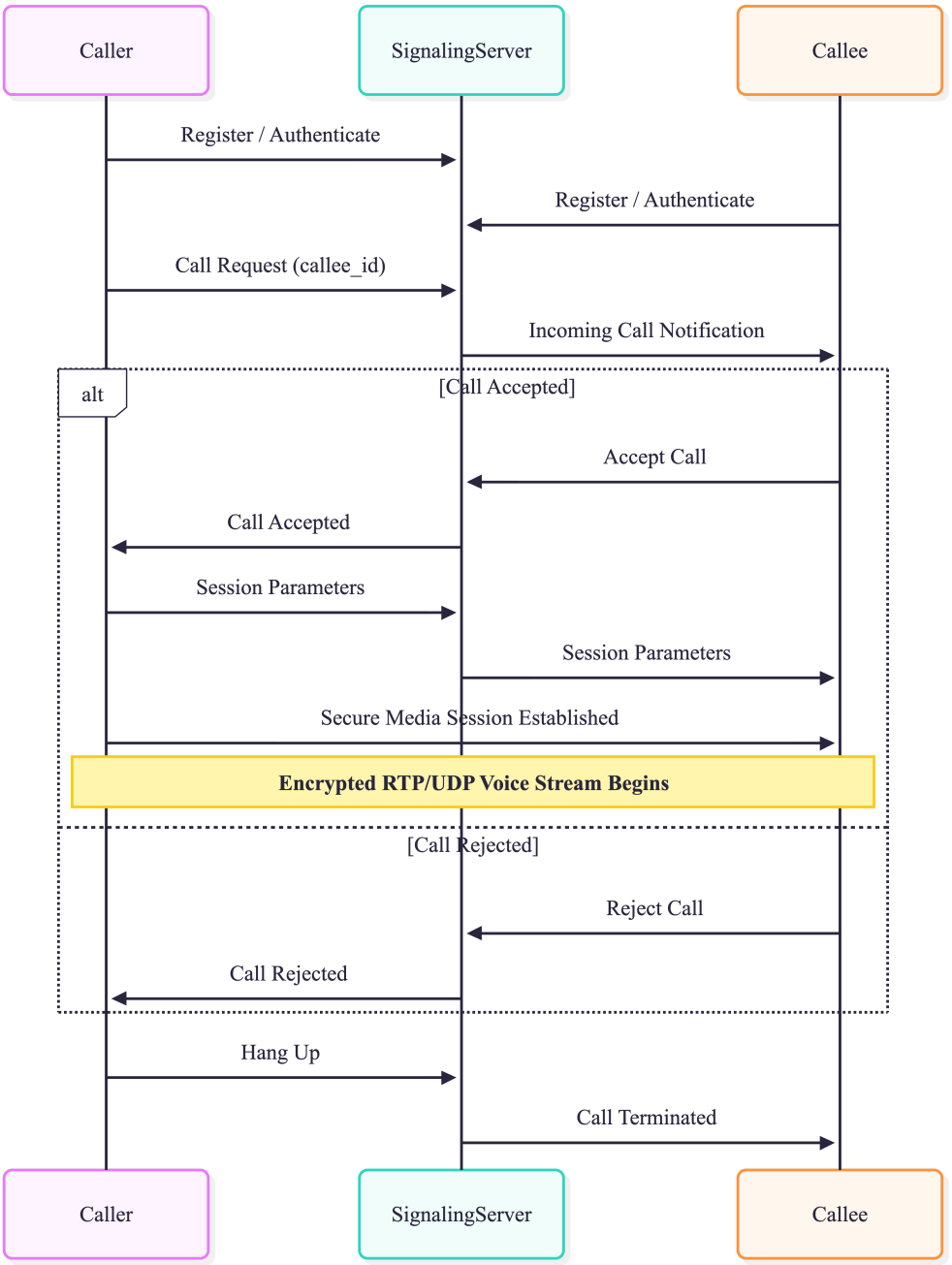


Figure 4-6: Signaling and Call Session Management Sequence

4.4.1 User Registration and Presence Management

Before initiating or receiving calls, users must register with the signaling server. During registration, user identity information and connection details are associated with an active session.

The signaling server maintains user presence information, allowing the system to determine whether a user is online, offline, or currently engaged in another call.

This information enables efficient call routing and availability checking.

4.4.2 Call Initiation

When a user initiates a call, the client application sends a call request to the signaling server containing the intended recipient's identifier.

The signaling server validates the request, verifies the recipient's availability, and forwards an incoming call notification to the destination user.

At this stage, no voice packets are transmitted. The system is solely responsible for establishing communication intent between the participating users.

4.4.3 Call Acceptance and Rejection

Upon receiving an incoming call notification, the recipient may either accept or reject the call.

If the call is rejected, the signaling server notifies the caller and the session establishment process is terminated.

If the call is accepted, both parties proceed to session establishment and communication parameter exchange.

4.4.4 Session Establishment

Following call acceptance, the signaling subsystem facilitates the exchange of session-related information required for voice communication.

This information includes network endpoints, encryption parameters, session identifiers,

and other metadata necessary for secure communication.

Once both participants have successfully exchanged and validated the required information, a communication session is established.

4.4.5 Call Maintenance

Throughout the duration of the call, the signaling subsystem remains responsible for managing session state and control messages.

Typical operations include participant status monitoring, connection supervision, call hold requests, and network state updates.

These control functions ensure that the communication session remains synchronized between participants.

4.4.6 Call Termination

A call session may be terminated by either participant. When a termination request is issued, the signaling server informs the remote participant and updates the session state accordingly.

Upon termination, all communication resources, temporary session information, and active media channels associated with the call are released.

This process ensures proper cleanup and prepares the system for future communication sessions.

4.4.7 Relationship with Media Transmission

The signaling subsystem is responsible only for call control and session coordination. Once a call session has been successfully established, real-time voice communication is performed by the media transmission subsystem.

As a result, the signaling server manages the call lifecycle while the sender-side, network transmission, and receiver-side components handle the actual voice data exchange.

4.5 Audio Deepfake Detection Service

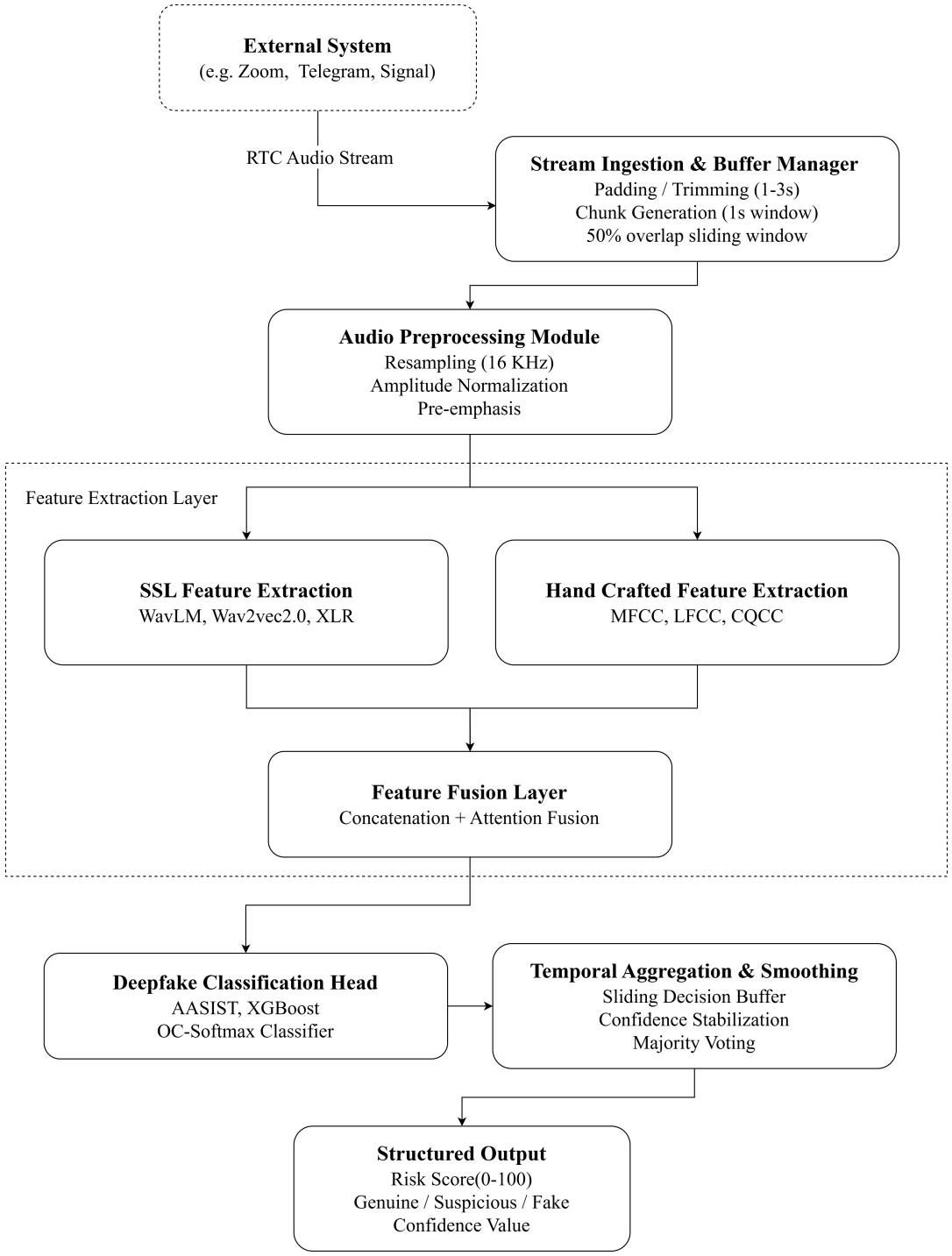


Figure 4-7: Audio Deepfake Detection Service

The audio deepfake detection service is the core inference component of the proposed system. It receives reconstructed receiver-side audio and performs analysis to determine whether the speech segment is real or synthetic. In contrast to tightly coupled end-to-end RTC implementations, the proposed design isolates deepfake detection as an independent service that can be integrated into different RTC/VoIP platforms through a standard API layer.

This service operates on short streaming windows rather than complete utterances, which makes it suitable for real-time deployment. Its output is a probabilistic decision that can be consumed by the RTC application for alert generation, monitoring, or downstream security enforcement.

4.5.1 Service Architecture

The detection module is implemented as a stateless inference service exposed through gRPC or HTTP APIs. This architecture allows the RTC system to send continuous stream to the detector without requiring modifications to the core communication stack.

Let the reconstructed receiver-side audio be denoted as $x_{out}[n]$.

The RTC client or receiver component forwards successive audio chunks to the detection service, which processes each chunk independently while preserving limited contextual information for temporal stability. The service can therefore be deployed in different environments, including:

- centralized cloud servers,
- edge nodes close to the RTC application,
- embedded deployment within a monitored communication platform.

This separation improves modularity, scalability, and maintainability. It also allows the deepfake detector to be updated independently of the RTC transmission pipeline.

4.5.2 Stream Ingestion and Buffer Manager

Because RTC systems operate on continuous audio streams, the detection service processes audio in short overlapping windows rather than waiting for an entire conversation to complete. Let the incoming reconstructed stream be segmented into windows of

length L with overlap O.

The streaming input can be written as

$$X = \{x_1, x_2, \ldots, x_T\}, \tag{4.11}$$

where each x_t represents a short audio chunk extracted from $x_{out}[n]$.

The chunking strategy serves two purposes:

- It reduces inference latency by enabling online processing,
- It preserves local temporal context needed for reliable spoof detection.

Overlapping windows are used to reduce instability near chunk boundaries and to ensure that short-lived artifacts are not missed due to segmentation. In practice, the detector produces one prediction per chunk, and these predictions are later aggregated into a stable decision.

4.5.3 Audio Preprocessing Module

Before feature extraction, each input chunk is normalized and prepared for model inference. This preprocessing stage is lightweight and deterministic so that it does not introduce unnecessary latency.

Typical preprocessing operations include:

- **Resampling** to a fixed sampling rate (16KHz) to match the training data.
- **Amplitude Normalization** standardizes the volume levels.
- **Pre-emphasis** applies a first order high-pass filter to amplify high frequency components.
- Trimming of silence or non-speech padding to mitigate variations caused by different microphones or sender side gain control,
- fixed-length padding for short segments.

Pre-emphasis is critical because RTC codecs often supress high frequencies yet these bands often contain the subtle spectral cues left by synthetic vocoders.

The result is a standardized audio segment suitable for feature extraction:

$$\tilde{x}_t = P(x_t), \tag{4.12}$$

where $P(\cdot)$ denotes the preprocessing function and $\tilde{x}_t$ is the normalized input chunk.

4.5.4 Feature Extraction Pipeline

The feature extraction stage converts the preprocessed waveform into representations that expose differences between real and synthetic speech. In the proposed framework, the system extracts two complementary views of audio simultaneously.

Let

$$z_t = F(\tilde{x}_t), \tag{4.13}$$

where $F(\cdot)$ denotes the feature extraction function and z_t is the resulting feature embedding.

The feature extraction module includes the following components:

- **Hand-Crafted Acoustic Features:** Traditional acoustic descriptors are extracted to capture low-level spectral and temporal characteristics of speech. These features include Mel-Frequency Cepstral Coefficients (MFCCs), spectral centroid, spectral bandwidth, zero-crossing rate, short-time energy, and pitch-related descriptors. Such features provide information about frequency distribution, voice quality, and temporal variations that may reveal artifacts introduced by speech synthesis and vocoding systems.
- **Self-Supervised Speech Embeddings:** High-level contextual speech representations are extracted using pretrained self-supervised learning (SSL) models such as WavLM, or Wav2Vec 2.0. These models generate rich embeddings that capture long-range acoustic and linguistic information while remaining robust to noise, codec compression, and transmission distortions commonly observed in RTC environments.
- **Frame-Level Acoustic Representations:** The continuous speech signal is segmented into short analysis frames, and frame-level embeddings are generated for

each segment. These representations preserve local spectral dynamics, temporal transitions, and fine-grained acoustic patterns that may contain artifacts associated with synthetic speech generation.

- **Phoneme-Aligned Feature Aggregation:** To improve linguistic consistency modeling, frame-level features are aggregated according to estimated phoneme boundaries. This process groups acoustically related frames into linguistically meaningful units, enabling the system to capture inconsistencies between phonetic content and acoustic realization that are often observed in deepfake speech.

By combining hand-crafted acoustic descriptors with self-supervised contextual representations, the proposed framework captures complementary information from both low-level signal characteristics and high-level speech structure.

Self-supervised embeddings are particularly useful because they capture high-level speech structure and remain relatively robust under codec compression, packet loss, and receiver-side enhancement. Phoneme-aligned aggregation further reduces sensitivity to frame-level noise by summarizing features over more stable linguistic units.

The detector is not relying solely on raw waveform patterns; it operates on representations that retain both acoustic detail and contextual speech information.

4.5.5 Classification Model Design

The classification module is responsible for determining whether a reconstructed RTC audio segment corresponds to real human speech or AI-generated synthetic speech. To evaluate both detection performance and real-time deployment feasibility, two complementary classification approaches are employed: AASIST and XGBoost.

For each incoming audio chunk x_t, a feature representation z_t is first extracted through the feature extraction pipeline. The resulting representation is then forwarded to a classification model that estimates the probability that the observed speech segment is spoofed.

The classification process is defined as

$$s_t = C(z_t), \tag{4.14}$$

where z_t denotes the extracted feature representation, $C(\cdot)$ represents the classifier, and $s_t \in [0, 1]$ is the spoofing probability assigned to audio chunk t.

AASIST-Based Classification

AASIST (Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks) serves as the primary deep learning detector in the proposed framework. Unlike conventional architectures that process temporal and spectral information separately, AASIST models speech as a heterogeneous graph and learns relationships between temporal and frequency-domain representations through graph attention mechanisms.

Given an input audio segment x_t, AASIST directly extracts discriminative spoofing representations and produces a spoofing score

$$s_t^{AASIST} = \mathcal{A}(x_t), \tag{4.15}$$

where $\mathcal{A}(\cdot)$ denotes the AASIST network. The model learns artifacts introduced by Text-to-Speech (TTS), Voice Conversion (VC), and other speech synthesis techniques, making it highly effective for audio deepfake detection. Due to its state-of-the-art performance on ASVspoof benchmarks and relatively compact architecture, AASIST is selected as the primary detector for evaluating robustness under RTC-induced degradations such as codec compression, packet loss, jitter buffering, and packet loss concealment.

XGBoost-Based Classification

XGBoost (Extreme Gradient Boosting) is employed as a computationally efficient baseline model. Unlike AASIST, which operates directly on waveform representations, XGBoost utilizes handcrafted acoustic features extracted from reconstructed RTC audio streams.

For each audio chunk, feature vectors consisting of Mel-Frequency Cepstral Coefficients (MFCCs), spectral statistics, energy-related descriptors, and other acoustic characteristics are extracted and represented as

$$z_t = [f_1, f_2, \ldots, f_n]. \tag{4.16}$$

The feature vector is then provided to the XGBoost classifier:

$$s_t^{XGB} = \mathcal{X}(z_t), \tag{4.17}$$

where $\mathcal{X}(\cdot)$ denotes the trained XGBoost model. The classifier generates a spoofing probability based on an ensemble of gradient-boosted decision trees.

XGBoost is included because previous studies have demonstrated that it achieves strong detection performance while maintaining extremely low inference latency. Consequently, it provides an effective benchmark for assessing the trade-off between detection accuracy and real-time computational efficiency.

4.5.6 Temporal Aggregation and Decision Stabilization

Since inference is performed on successive streaming chunks, individual predictions may fluctuate due to short-term artifacts, transient packet loss, or local noise variation. To improve decision stability, the proposed system applies temporal aggregation over a sliding window of recent predictions.

Let the set of recent spoofing scores be

$$S_t = \{s_{t-k+1}, s_{t-k+2}, \ldots, s_t\}, \tag{4.18}$$

where k is the aggregation window size.

A smoothed score can be computed as

$$\bar{s}_t = A(S_t), \tag{4.19}$$

where $A(\cdot)$ denotes the aggregation function. In implementation, this may be a moving average, weighted average, or majority-based smoothing rule.

The final decision is then obtained from the aggregated score:

$$\hat{d}_t = \begin{cases} 1, & \bar{s}_t \geq \tau \\ 0, & \bar{s}_t < \tau \end{cases} \tag{4.20}$$

This stabilization step reduces false alarms caused by isolated distortions and produces a more reliable real-time output for the RTC application.

4.5.7 Output Interface and Response Format

The detection service returns a structured output to the RTC system through the same API channel used for audio ingestion. The response contains both a continuous risk score and a discrete decision label.

A typical output contains:

- spoofing probability score s_t,
- binary class label $\hat{d}_t$,
- optional confidence value,
- optional timestamp or chunk identifier.

The output can therefore be represented as

$$R_t = \{s_t, \hat{d}_t, \text{timestamp}_t\}. \tag{4.21}$$

This response enables real-time actions such as UI alerts, call monitoring warnings, logging for security analysis, and policy-based intervention by the RTC platform.

4.6 Integration Strategy with RTC Systems

The integration strategy defines how the proposed audio deepfake detection service is connected to real-world real-time communication (RTC) environments without requiring modifications to the core RTC application. Since many RTC platforms, including WebRTC-based systems, VoIP services, and conferencing applications, operate as closed or partially restricted ecosystems, the proposed detector is designed as an external service-oriented component that interfaces with the RTC pipeline through standardized audio streaming access points.

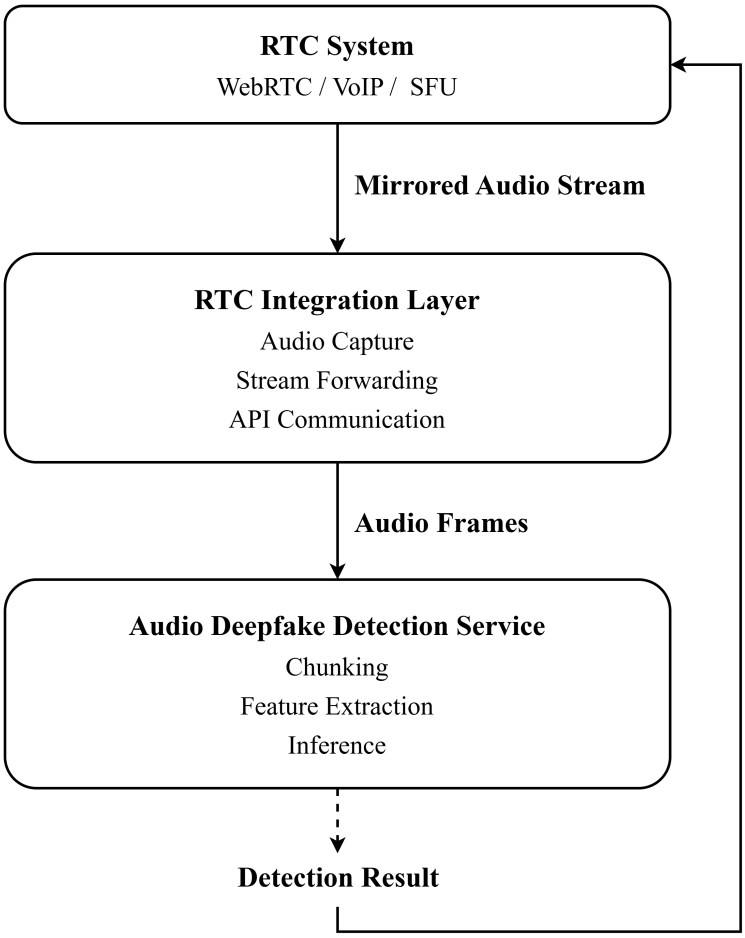


Figure 4-8: Integration of Detection Service with RTC systems

The primary objective of this integration layer is to enable continuous detection on reconstructed audio streams while preserving the real-time properties of the communication system. The detector is therefore deployed as a parallel inference component rather than as an in-band modification of the RTC stack.

4.6.1 Deployment Modes

Two deployment configurations are considered for integrating the detection service into RTC environments.

In the **client-side deployment**, a lightweight agent or SDK is embedded within the developed RTC application environment. This agent captures reconstructed audio on the receiver side and forwards audio chunks to the detection service for inference. This approach can provide fast response times and low network overhead; however, it is

limited by device compute capacity, deployment complexity, and privacy constraints.

In the **server-side deployment**, which is the preferred configuration in this work, audio is intercepted at the media server or selective forwarding unit (SFU) level after decoding and jitter-buffer processing. The detection service then consumes mirrored audio streams centrally, enabling consistent processing across multiple sessions while avoiding the computational burden on end-user devices. This approach is particularly suitable for enterprise RTC systems and server-managed communication platforms.

4.6.2 Integration Point in the RTC Pipeline

The proposed detector is attached at the stage where audio has already been reconstructed into a stable waveform representation. This point is typically located after receiver-side decoding and enhancement, or at the output of a media server that forwards a mirrored copy of the stream.

The detector operates as a passive observer of the reconstructed audio stream and does not alter the primary communication path. This design ensures that deepfake analysis does not degrade call quality or introduce interference in the RTC session.

Technically, the integration may be realized using RTP stream duplication, media server hooks, audio tap points, or WebRTC-compatible plugin mechanisms, depending on the target deployment environment.

4.6.3 API-Based Communication Workflow

Communication between the developed RTC system and the detection service is handled through a streaming API layer, typically implemented using gRPC for low-latency environments or HTTP-based streaming for simpler deployments.

The workflow follows a chunk-based streaming model in which reconstructed audio segments are transmitted to the detection service in near real time. Each chunk is processed independently, while a small temporal buffer is maintained to preserve continuity across adjacent windows.

Let the reconstructed audio stream be represented as $x_{out}[n]$. The integration layer

segments this stream into chunks

$$X = \{x_1, x_2, \ldots, x_T\}, \tag{4.22}$$

which are transmitted to the detector through the API channel.

For each input chunk, the service returns an inference result consisting of a spoofing probability and an associated class label. This request-response design keeps the detector platform-agnostic and avoids dependency on specific codec or transport implementations.

4.6.4 Scalability and Latency Considerations

RTC applications impose strict latency requirements, so the detection system is designed to operate under bounded inference delay. To maintain responsiveness, the service processes fixed-size audio chunks instead of full utterances and uses compact feature extractors at runtime.

Scalability is achieved through horizontal replication of the detection service behind a load balancer. This allows multiple concurrent RTC sessions to be processed in parallel without significant performance degradation. To further reduce runtime overhead, heavy feature extraction components may rely on pretrained encoders that remain frozen during inference. This design balances detection accuracy with real-time constraints and ensures that the system can operate reliably in live RTC environments.

4.7 Evaluation Metrics and Performance Assessment

Since the system is designed for integration within RTC/VoIP environments, evaluation considers both detection accuracy and computational efficiency and latency constraints.

The performance evaluation is therefore divided into two categories:

4.7.1 Detection Performance Metrics

The primary objective of the proposed framework is to accurately distinguish real speech from synthetic speech under realistic RTC transmission conditions. To assess classification performance, the following metrics are employed.

Equal Error Rate (EER) is the primary evaluation metric widely adopted in audio spoofing and deepfake detection research, particularly in ASVspoof benchmarks. EER represents the operating point at which the False Acceptance Rate (FAR) equals the False Rejection Rate (FRR).

A lower EER indicates better discrimination capability between real and spoofed speech samples.

$$EER = FAR = FRR \tag{4.23}$$

Since EER is threshold-independent and directly reflects spoof detection performance, it serves as the principal metric for model comparison.

Receiver Operating Characteristic Area Under Curve (ROC-AUC) measures the ability of the classifier to distinguish between real and synthetic speech across all possible decision thresholds.

The ROC curve plots:

- True Positive Rate (TPR)
- False Positive Rate (FPR)

The Area Under Curve (AUC) is then computed as:

$$AUC = \int_0^1 TPR(FPR) \times d(FPR) \tag{4.24}$$

Higher ROC-AUC values indicate stronger classification capability and better threshold robustness.

Precision measures the proportion of detected deepfake samples that are actually synthetic.

$$Precision = \frac{TP}{TP + FP} \tag{4.25}$$

where:

- TP = True Positives
- FP = False Positives

High precision indicates a low false alarm rate.

Recall measures the proportion of actual synthetic speech samples correctly detected by the system.

$$Recall = \frac{TP}{TP + FN} \tag{4.26}$$

where:

- FN = False Negatives

High recall indicates strong detection sensitivity.

F1-Score provides a balanced measure of precision and recall and is particularly useful when class distributions are imbalanced.

$$F1 = 2 \cdot \frac{Precision \times Recall}{Precision + Recall} \tag{4.27}$$

Higher F1-scores indicate improved overall classification effectiveness.

Confusion Matrix Analysis is used to analyze prediction behavior by reporting:

- True Positives (TP)
- True Negatives (TN)
- False Positives (FP)
- False Negatives (FN)

This analysis provides insight into the types of classification errors made by the detection system.

4.7.2 Real-Time System Performance Metrics

Since the proposed framework targets live RTC environments, computational efficiency and responsiveness are critical deployment requirements.

Inference Latency measures the time required for processing an incoming audio segment and producing a prediction.

$$Latency = t_{output} - t_{input} \tag{4.28}$$

Low inference latency is essential for maintaining real-time operation and minimizing delay in user-facing applications.

Throughput measures the number of audio segments or audio streams processed per unit time.

$$Throughput = \frac{Number\ of\ Processed\ Chunks}{Unit\ Time} \tag{4.29}$$

This metric evaluates the scalability of the proposed detection service under increasing workloads.

Memory Utilization measures the amount of system memory consumed during inference.

This metric is important for deployment on resource-constrained environments such as edge servers, embedded systems, or cloud instances with limited resources.

CPU and GPU Utilization utilization are monitored during model inference to assess computational efficiency and hardware requirements.

These measurements help determine the practical feasibility of deploying the proposed framework in real-time communication infrastructures.

4.8 Experimental Setup and Evaluation Protocol

This section outlines the experimental methodology that will be used to evaluate the proposed Audio Deepfake Detection (ADD) framework under realistic RTC communication environments. The evaluation protocol is designed to assess both detection effectiveness and deployment feasibility under various transmission conditions commonly encountered in VoIP and RTC systems.

The experiments will focus on three primary aspects: robustness against RTC-induced distortions, comparative analysis of multiple detection architectures, and assessment of computational feasibility for real-time deployment.

4.8.1 Dataset Configuration

The experimental evaluation will primarily utilize benchmark datasets from the ASVspoof challenge series and other publicly available audio deepfake corpora.

RTCFake (2025): The first large-scale dataset specifically tailored for RTC, totaling approximately 600 hours It features precise "offline-online" pairing, created by transmitting speech through seven mainstream platforms: Zoom, WeChat, QQ, DingTalk, Lark, VooV, and Telegram. It includes both seen and unseen platforms and complex noise conditions for evaluating generalization

ADD-C (2024): A benchmarking test dataset designed to assess robustness under diverse communication conditions. It simulates six widely used VoIP/VoLTE codecs (AMR-WB, EVS, IVAS, OPUS, Speex, and SILK) across five different Packet Loss Rates (PLRs) ranging from 0% to 20%.

Teams Meeting Dataset (2024): A dataset curated specifically for real-time testing in communication platforms using actual Microsoft Teams group meeting sessions. It includes authentic voice samples and synthetic events introduced via voice cloning and conversion, with trials lasting 10 seconds each.

ASVspoof 5 (2024/2026): Focuses on "Generic Telephony or VoIP" scenarios. It incorporates crowdsourced data recorded in diverse acoustic conditions and processes it with both conventional and contemporary neural codecs (e.g., EnCodec) to simulate modern VoIP quality.

ASVspoof 2021 (DF Task): An earlier but foundational dataset that introduced codec and compression artifacts to evaluate generalizability toward unknown channel variations in real telephone systems.

A subset of our evaluation data will consist of paired "offline-online" samples. This involves playing clean synthetic and real audio through a virtual device and recording the output after it has been processed by the internal "black-box" pipelines (noise suppression, echo cancellation, and encryption) of platforms like Zoom and Microsoft Teams.

The dataset will be partitioned into three speaker-disjoint subsets:

Training Set: consist balanced real/fake utterances augmented with simulated codecs and packet loss

Development Set: used for hyperparameter tuning and setting detection thresholds

Evaluation Set: will include unseen spoofing algorithms and unseen communication platforms to rigorously test the system's cross-domain generalization and real-time performance

4.8.2 Synthetic Speech Generation Methods

The evaluation datasets used in this study contain synthetic speech generated using multiple state-of-the-art speech synthesis techniques. These techniques represent the primary attack categories encountered in modern audio deepfake systems.

The synthetic speech samples include:

- **Text-to-Speech (TTS):** Systems that generate speech directly from text input using neural speech synthesis architectures.
- **Voice Conversion (VC):** Systems that transform the voice characteristics of a source speaker into those of a target speaker while preserving linguistic content.
- **Neural Voice Cloning:** Systems that synthesize speech resembling a target speaker using a limited amount of reference audio.
- **Modern Generative Paradigms:** Recent advancements utilize Diffusion Models (e.g., Grad-TTS) and Neural Vocoders (e.g., HiFi-GAN, BigVGAN) to achieve high-fidelity, efficient waveform synthesis that is increasingly difficult for human listeners to distinguish from real speech

The selected datasets contain speech generated using a variety of synthesis architectures, including autoregressive, non-autoregressive, diffusion-based, and neural codec-based speech generation models.

4.8.3 RTC Distortion Robustness Evaluation

One of the primary objectives of this research is to investigate the impact of RTC communication channels on deepfake detection performance. Therefore, the proposed framework will be evaluated under multiple transmission conditions representing common VoIP environments.

The evaluation will consider variations in:

- Codec compression using Opus, G.711, SILK, and AMR-WB.
- Packet loss rates of 0%, 5%, 10%, 15%, and 20%.
- Network delay and jitter conditions representing stable and unstable communication channels.
- Bandwidth-constrained scenarios with adaptive bitrate operation.
- Combined impairment scenarios involving simultaneous codec compression, packet loss, and network variability.

By systematically varying these parameters, the study aims to determine the robustness of the proposed detection framework against realistic communication impairments.

4.8.4 Comparative Model Evaluation

The proposed study will investigate multiple deepfake detection architectures to identify the most suitable model for deployment in RTC environments.

Candidate models include:

- AASIST
- XGBoost-based classifiers

In addition, different feature extraction approaches will be explored using self-supervised speech representations such as WavLM and Wav2Vec 2.0.

All models will be evaluated using the performance metrics defined in Section 4.7. Comparative analysis will be performed to examine differences in detection accuracy, robustness to transmission distortions, and suitability for real-time deployment.

4.8.5 Computational Feasibility Assessment

Since the proposed framework is intended for integration within real-time communication systems, computational efficiency is an important consideration.

The experimental study will therefore include an assessment of runtime characteristics, including:

- Inference latency
- Throughput under concurrent audio streams
- Memory consumption
- CPU and GPU utilization
- Model size and deployment complexity

This analysis will help determine whether the proposed framework can satisfy the latency and scalability requirements of practical RTC applications.

4.8.6 Evaluation Objectives

The experimental protocol is designed to answer the following research questions:

1. How do RTC transmission distortions affect audio deepfake detection performance?
2. Which combination of feature extraction and classification architecture provides the highest robustness under RTC conditions?
3. What trade-offs exist between detection accuracy and computational efficiency?
4. Can the proposed framework satisfy the requirements of real-time deployment within RTC systems?

5. EXPECTED OUTCOMES

The proposed project is expected to achieve the following outcomes:

- A functional prototype system capable of detecting synthetic (AI-generated) speech in real-time from streaming audio inputs.
- A signal-processing and machine learning pipeline that extracts relevant speech features (e.g., MFCC, spectrogram-based representations, or SSL embeddings) for classification of real vs synthetic speech.
- A low-latency inference system designed to operate under VoIP-like communication conditions including codec compression, noise, and packet loss.
- Experimental evaluation results demonstrating system performance in terms of accuracy, robustness under degradation, and real-time processing latency.
- Comparative analysis of different feature extraction techniques and classification models for real-time deployment feasibility.
- A fully working system including VoIP communication degradation to test system behavior under realistic network conditions.
- A deployable prototype (API or local application) demonstrating end-to-end detection from audio stream input to classification output.

6. FEASIBILITY ANALYSIS

6.1 Technical Feasibility

The proposed Audio Deepfake Detection (ADD) system is technically feasible due to the availability of public datasets, deep learning frameworks, and audio processing tools. The system can be developed using Python, PyTorch, Librosa, and related libraries. Existing RTC technologies and audio simulation tools can be used to emulate real-world VoIP conditions such as codec compression, packet loss, and network delay. The required computational resources are available through local GPU systems or cloud platforms.

6.2 Operational Feasibility

The system is operationally feasible because it addresses a practical security concern in modern RTC and VoIP platforms. The proposed detector operates as an external service and can be integrated into existing communication systems without modifying their core functionality. This allows real-time monitoring while preserving normal communication workflows.

6.3 Economic Feasibility

The project requires minimal financial investment since most development tools, frameworks, and datasets are open source. The primary costs involve computing resources, internet access, and optional cloud infrastructure for model training and deployment. These costs are manageable within the scope of a student project.

6.4 Schedule Feasibility

The project can be completed within the allocated academic timeframe. Development activities including literature review, dataset preparation, model development, RTC simulation, system integration, evaluation, and documentation can be planned and executed incrementally throughout the project duration.

6.5 Ethical and Legal Feasibility

The project uses publicly available datasets and focuses on detecting synthetic speech for security and research purposes. No private voice data will be collected without consent. The proposed system is intended as a defensive tool to improve trust and security in communication systems and does not interfere with normal user communications.

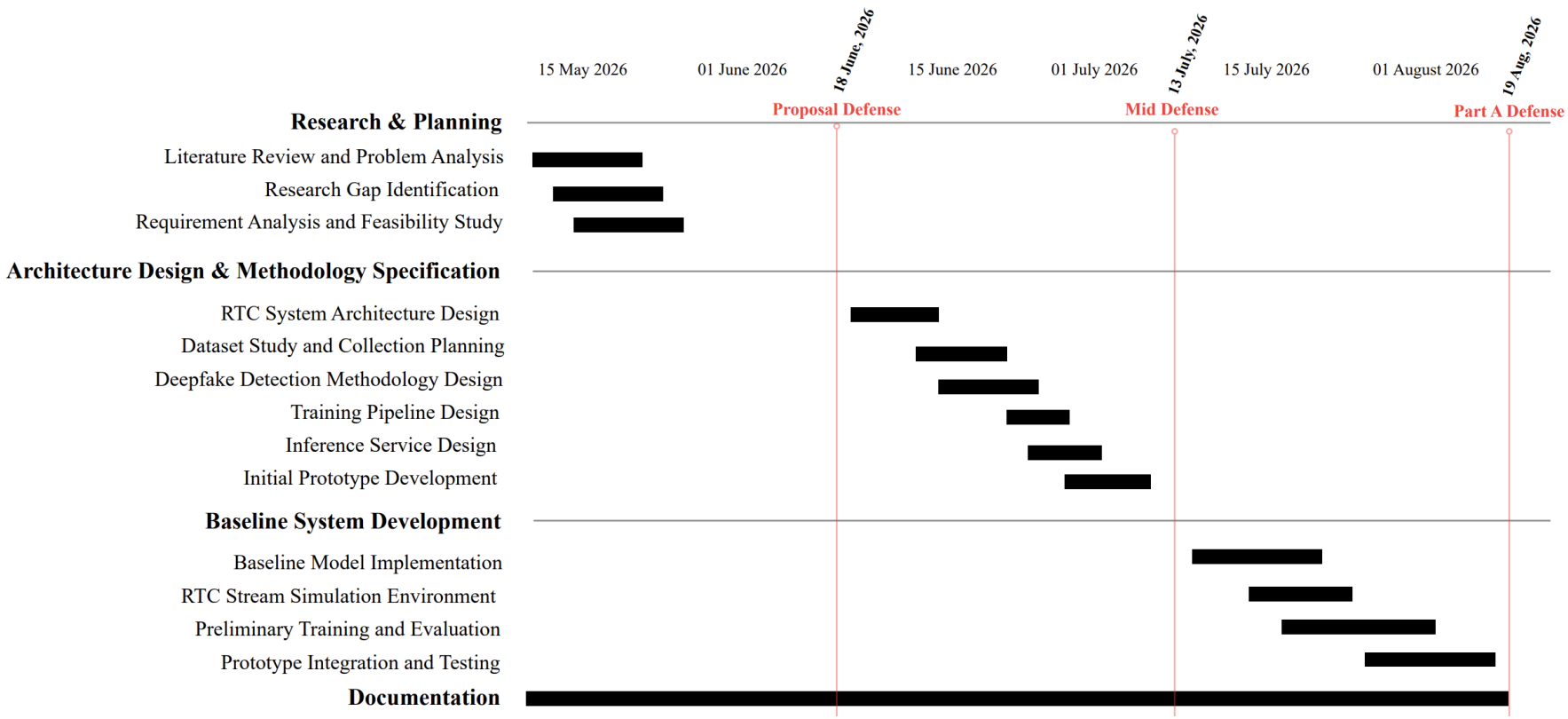
APPENDICES

Appendix A: Project Budget

Table A-1: Expected Project Budget

Particular	**Qty**	**Rate (NRs.)**	**Total**
External USB Microphone	2	2500	5000
Headphones for Audio Monitoring	2	1800	3600
Cloud GPU / Compute Subscription	1	8000	8000
Linux Server (VPS)	1	6000	6000
Documentation and Report Preparation	–	–	3000
Research, Experimentation and Miscellaneous	–	–	5000
Total Cost			**30,600**

Table B-1: Expected Project Timeline – Part A



References

[1] Y. Wang, R. Skerry-Ryan, D. Stanton, Y. Wu, R. J. Weiss, N. Jaitly, Z. Yang, Y. Xiao, Z. Chen, S. Bengio, *et al.*, "Tacotron: Towards end-to-end speech synthesis," *arXiv preprint arXiv:1703.10135*, 2017.

[2] R. Skerry-Ryan, E. Battenberg, Y. Xiao, Y. Wang, D. Stanton, J. Shor, R. Weiss, R. Clark, and R. A. Saurous, "Towards end-to-end prosody transfer for expressive speech synthesis with tacotron," in *International Conference on Machine Learning (ICML)*, pp. 4693–4702, PMLR, 2018.

[3] Y. Wang, D. Stanton, Y. Zhang, R.-S. Ryan, E. Battenberg, J. Shor, Y. Xiao, Y. Jia, F. Ren, and R. A. Saurous, "Style tokens: Unsupervised style modeling, control and transfer in end-to-end speech synthesis," in *International Conference on Machine Learning (ICML)*, pp. 5180–5189, PMLR, 2018.

[4] Y. Jia, Y. Zhang, R. Weiss, Q. Wang, J. Shen, F. Ren, P. Nguyen, R. Pang, I. Lopez-Moreno, Y. Wu, *et al.*, "Transfer learning from speaker verification to multi-speaker text-to-speech synthesis," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[5] S. R. Reddy, P. K. Sarangi, S. Nikhil, M. A. Jabbar, and S. Mekala, "A survey on deep fake audio detection using deep learning," in *Proceedings of the 1st International Conference on Research and Development in Information, Communication, and Computing Technologies (ICRDICCT'25)*, vol. 4, pp. 283–289, 2025.

[6] A. Raza, A. Basit, A. Amin, Z. A. Arfeen, M. I. Masud, U. Fayyaz, and T. A. Jumani, "A comprehensive review of deepfake detection techniques: From traditional machine learning to advanced deep learning architectures," *AI*, vol. 7, no. 1, pp. 1–42, 2026.

[7] J. Yamagishi, X. Wang, M. Todisco, M. Sahidullah, J. Patino, A. Nautsch, X. Liu, K. A. Lee, T. Kinnunen, N. Evans, *et al.*, "Asvspoof 2021: Accelerating progress in spoofed and deepfake speech detection," in *Proceedings of the ASVspoof Challenge Workshop*, pp. 47–54, 2021.

[8] M. Li, Y. Ahmadiadli, and X.-P. Zhang, "A survey on speech deepfake detection," *Frontiers in Big Data*, vol. 7, 2024.

[9] A. Pianese, D. Cozzolino, G. Poggi, and L. Verdoliva, "Deepfake audio detection by speaker verification," in *2022 IEEE International Workshop on Information Forensics and Security (WIFS)*, pp. 1–6, 2022.

[10] J. J. Mathew, R. Ahsan, S. Furukawa, J. G. K. Kumar, H. Pallan, A. S. Padda, S. Adamski, M. Reddiboina, and A. Pankajakshan, "Towards the development of a real-time deepfake audio detection system in communication platforms," *arXiv preprint arXiv:2403.11778*, 2024.

[11] J. Xue, Z. Yi, Y. Huang, Y. Ren, Y. Chen, C. Fan, Z. Su, Y. Zhang, and B. Cai, "Rtcfake: Speech deepfake detection in real-time communication," *arXiv preprint arXiv:2604.23742*, 2025.

[12] X. Wang, H. Delgado, H. Tak, J.-w. Jung, H.-j. Shim, M. Todisco, I. Kukanov, X. Liu, M. Sahidullah, T. Kinnunen, N. Evans, K. A. Lee, and J. Yamagishi, "Asvspoof 5: Crowdsourced speech data, deepfakes, and adversarial attacks at scale," *arXiv preprint arXiv:2408.08739*, 2024.

[13] H. Azzuni and A. El Saddik, "Voice cloning: A comprehensive survey," *arXiv preprint*, 2024.

[14] B. Zhang, H. Cui, V. Nguyen, and M. Whitty, "Audio deepfake detection: What has been achieved and what lies ahead," *Sensors*, vol. 25, no. 7, p. 1989, 2025.

[15] J. J. Bird and A. Lotfi, "Real-time detection of ai-generated speech for deepfake voice conversion," *arXiv preprint arXiv:2308.12734*, 2023.

[16] Y. Shi *et al.*, "Benchmarking audio deepfake detection: Robustness under real-world communication scenarios," in *Proceedings of Interspeech*, 2024.

[17] L. Pham, A. Schindler, F. Skopik, P. Lam, T. Nguyen, H. Tang, D. Tran, A. Polonsky, and C. Vu, "A comprehensive survey with critical analysis for deepfake speech detection," *arXiv preprint*, 2024.

[18] X. Zhang, Z. Zhang, Y. Wang, L. Li, L. Jin, and M. Li, "Multiapi spoof: A multi-api dataset and local-attention network for speech anti-spoofing detection," *arXiv preprint arXiv:2512.07352*, 2025.

[19] X. Wang, H. Delgado, N. Evans, X. Liu, T. Kinnunen, H. Tak, K. A. Lee, I. Kukanov, M. Sahidullah, M. Todisco, and J. Yamagishi, "Asvspoof 5: Evaluation of spoofing, deepfake, and adversarial attack detection using crowdsourced speech," *Computer Speech & Language*, vol. 95, p. 101825, 2026.

[20] J. Wu, W. Lu, X. Luo, R. Yang, Q. Wang, and X. Cao, "Coarse-to-fine proposal refinement framework for audio temporal forgery detection and localization," in *Proceedings of the 32nd ACM International Conference on Multimedia (MM '24)*, pp. 7395–7403, 2024.

[21] M. Li, Y. Ahmadiadli, and X.-P. Zhang, "Audio anti-spoofing detection: A survey," *arXiv preprint arXiv:2404.13914*, 2024.

[22] A. Firc, K. Malinka, and P. Hanáček, "Deepfakes as a threat to speaker and facial recognition: An overview of tools and attack vectors," *Heliyon*, vol. 9, no. 4, p. e15090, 2023.